2025

📝 Question 1 (10 pts)

英文題目：

True or False. Please indicate the error and correct it if it is false.

中文翻譯：

判斷下列敘述為「正確（True）」或「錯誤（False）」，若為錯誤，請指出錯誤之處並寫出正確說法。

◆ (a)

英文題目：

We say that a data object of Type B HAS-A data object of Type A if A is more specified than B.

中文翻譯：

若 A 比 B 更具體（更特化），則我們說 Type B 擁有（HAS-A）一個 Type A 的資料物件。

🔍詳細分析

本題考的是物件導向中的 **HAS-A（組合關係）與 IS-A（繼承關係）區別**。HAS-A 的核心概念是「包含關係」，也就是某個物件內部擁有另一個物件，例如 Car HAS-A Engine，表示車子裡面包含引擎。而題目卻用「A 比 B 更具體」來判斷 HAS-A，這其實是混淆了繼承（IS-A）與組合（HAS-A）。「更具體」通常描述的是類別之間的階層關係（例如 Dog is-a Animal），而不是物件內部的組成結構。因此題目邏輯錯誤。

✅答案

✗ False

✓ 正確說法：

Type B HAS-A Type A 表示

☞ **B contains A / A is a part of B（B 裡面包含 A）**

◆ (b)

英文題目：

A derived class can inherit all the non-private members (data and functions) of the base class including constructor and destructor.

中文翻譯：

衍生類別可以繼承基底類別所有非 private 成員（資料與函式），包含建構子與解構子。

🔍詳細分析

本題考的是 C++ 中的 **繼承機制**。一般來說，derived class 確實可以繼承 base class 的 public 和 protected 成員，但 constructor 和 destructor 並不屬於一般可被繼承的成員。建構子是用來初始化物件的，每個類別都需要自行定義自己的建構流程；解構子則負責釋放資源，也不能被直接繼承。雖然 derived class

可以「呼叫」base class 的 constructor（透過 initializer list），但這不等於「繼承」。因此題目將 constructor 與 destructor 誤列為可繼承成員，是錯誤的。

✅答案

✗ False

✓ 正確說法：

Derived class

☞ **cannot inherit constructors and destructors**

◆ (c)

英文題目：

We need to overload operator << in a user defined class so that the corresponding object can be output by using cout. The definition of operator must be of a class member function.

中文翻譯：

我們需要在使用者自訂類別中多載 << 運算子，使物件可以透過 cout 輸出，而該運算子必須定義為類別的成員函式。

🔍詳細分析

本題考的是 **C++ 運算子多載（operator overloading）**。當我們使用 cout << obj 時，左邊的運算元其實是 cout（型別為 ostream），而不是自訂類別物件。因此若將 operator<< 寫成類別成員函式，語法上會變成 obj << cout，這與實際使用方式不符。正確做法是將 operator<< 定義為「非成員函式」，通常會搭配 friend function 以便存取類別內部資料。因此題目說「必須是 class member function」是錯誤的。

✅答案

✗ False

✓ 正確說法：

operator<< 應定義為

☞ **non-member function（通常為 friend function）**

◆ (d)

英文題目：

When we want to solve a SAT problem, it must be implemented by a traversing algorithm in the preorder sequence.

中文翻譯：

當我們要解 SAT 問題時，必須使用前序走訪（preorder traversal）來實作。

🔍詳細分析

本題考的是 **演算法概念的區分**。SAT（Boolean Satisfiability Problem）是一個經典的 NP-complete 問題，通常使用回溯法（backtracking）、DPLL 或 CDCL 等搜尋型演算法來解決。這些方法的核心在於系統性地嘗試不同的變數配置，而不是單純地「走訪一棵樹」。preorder traversal 是一種樹的走訪方式，用於訪問節點順序，但並不是解 SAT 的必要或標準方法。因此題目將 traversal 與解題策略混為一談，是錯誤的觀念。

✅答案

✕ False
✓ 正確說法：
SAT 問題通常使用
☞ **backtracking / search algorithms，而非 preorder traversal**

◆ (e)

英文題目：
Nested classes are preferred than composite classes since a single node class can be used by other classes.
中文翻譯：
巢狀類別（nested classes）比組合類別（composite classes）更好，因為單一節點類別可以被其他類別使用。

🔍詳細分析

本題考的是 **物件導向設計（OOP design）中的重用性與結構設計**。Nested class 是定義在另一個類別內部的類別，通常只供外層類別使用，因此可重用性較低；相反地，composite class（組合類別）是獨立存在的，可以被多個不同類別共同使用，具有較高的模組化與重用性。因此題目說 nested class 比較好，理由又是「可以被其他類別使用」，這與實際設計原則相反。

✅答案

✕ False
✓ 正確說法：
Composite classes are preferred
☞ **when a class needs to be reused by multiple classes**

以下是**第 2 題完整（英文原題＋中文翻譯＋詳細分析＋答案整合）**，我已幫你整理成**期中考可以直接寫的高分版本（詳細解析）**👇

📝 Question 2 (5 pts)

📘 英文題目

Please give the space complexity (i.e., recursion stack space) of the binary search algorithm in the following using the asymptotic notation and explain it shortly.

📗 中文翻譯

請求出以下二元搜尋（binary search）演算法的**空間複雜度（特別是遞迴呼叫所使用的 stack 空間）**，並用漸進符號（asymptotic notation）表示，同時簡要說明原因。

```
int BinarySearch (int *a, const int x, const int left,
        const int right)
{// Search the sorted array a[left], ... , a[right] for x
   if (left <= right) {
      int middle = (left + right)/2;
      if (x < a[middle]) {
         return BinarySearch(a, x, left, middle-1);
      } else if (x > a[middle]) {
         return BinarySearch(a, x, middle+1, right);
      } else return middle;
   } //end of if
   return -1; //not found
}
```

```
void DblList::Insert(DblListNode *p,
DblListNode *x) {
// insert node p to the right of node x
   p->left = x; p->right = x->right;
   x->right->left = p; x->right = p;
}
```

🔍 中文詳細分析（考試重點）

這題其實在考兩個關鍵觀念：

☞ 1 **Binary Search 的遞迴結構**

☞ 2 **空間複雜度（stack space）如何計算**

□ (1) Binary Search 遞迴特性

題目中的程式（第 6 頁圖）顯示：

- 每次會計算 middle
- 然後只往「一半區間」繼續搜尋

☞ 也就是：

每一層遞迴都把問題規模減半

□ (2) 遞迴深度（最重要🔥）

假設資料大小是 n

每次變化：

n → n/2 → n/4 → n/8 → ...

☞ 最多能切幾次？
☞ 直到剩下 1
所以：
☞ 遞迴層數 = $\log_2(n)$

□ (3) 空間複雜度怎麼算

每一次遞迴呼叫都會佔用一個 stack frame
☞ stack 使用量 = 遞迴深度
因此：
☞ 空間複雜度 = O(log n)

❗ 常見錯誤（老師很愛考）

✕ 很多人會寫：O(1)
☞ 那是「非遞迴版本」
✕ 但題目強調：
☞ recursion stack space
☞ 所以一定是：
☞ **O(log n)**

✍ 完整答案（可直接寫）

The space complexity of the recursive binary search algorithm is **O(log n)**.
This is because each recursive call reduces the search space by half, resulting in a recursion depth of at most $\log_2(n)$. Since each recursive call consumes stack space, the total recursion stack usage is proportional to the recursion depth. Therefore, the space complexity is O(log n).

📌 中文完整答案版本（申論直接寫）

此題要求的是二元搜尋在「遞迴情況下的空間複雜度」。由於 binary search 每一次遞迴都會將搜尋範圍縮小為原來的一半，因此問題規模會從 n 變為 n/2、n/4、n/8……直到縮小為 1。這樣的過程最多會進行 $\log_2(n)$ 次，因此遞迴的深度為 O(log n)。由於每一層遞迴都會佔用一個函式呼叫的 stack 空間，所以整體的空間使用量與遞迴深度成正比，因此空間複雜度為 O(log n)。

🔥 最後一句（必背）

☞ **Binary Search（recursive）空間複雜度 = O(log n)**
☞ **原因：遞迴深度 = log n**

以下是**第 3 題完整整理（英文原題＋中文翻譯＋詳細分析＋答案整合）**，我幫你整理成**期中考可直接寫的高分版本（完整解釋超過百字）**👇

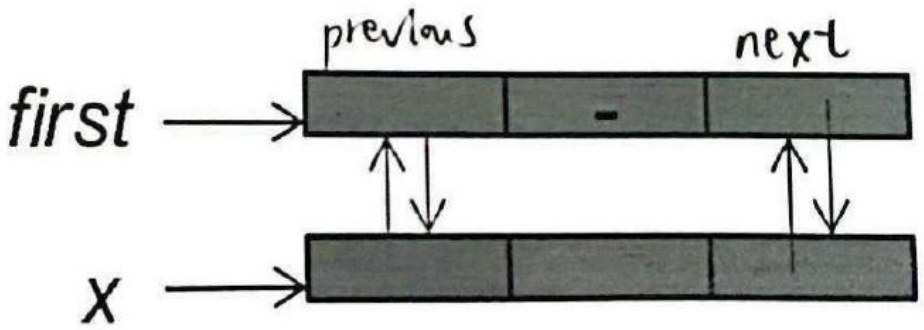
📝 Question 3 (5 pts)

📘 英文題目

Please show the result after we insert a new node p behind the node x in the following data structure.

📗 中文翻譯

請畫出在以下資料結構中，當我們在節點 x 的後面插入一個新節點 p 之後的結果。



🔍 中文詳細分析（考試重點）

這題是在考 **雙向鏈結串列（Doubly Linked List）插入操作**🔥

從題目圖（第 7 頁）可以看出：

- 有一個節點 x
- x 有：
  - prev（指向前一個節點）
  - next（指向下一個節點）

👉 題目要求：

在 **x 的後面插入新節點 p**

□ 插入核心觀念（超重要🔥）

在 doubly linked list 中插入節點時：

👉 不能只改一條指標

👉 必須「4 個指標全部更新」

□ 插入步驟（邏輯）

假設：

- x → 原本的下一個節點是 y

插入 p 之後變成：
x → p → y

🔧 指標更新順序（考試必寫）

1 p->left = x
☞ p 的前一個指向 x
2 p->right = x->right
☞ p 的下一個指向原本的 y
3 x->right->left = p
☞ 原本 y 的前一個改成 p
4 x->right = p
☞ x 的下一個改成 p

✍ 完整答案（可直接寫）

After inserting node p behind node x, node p will be placed between node x and the original next node of x. The pointers must be updated carefully to maintain the structure of the doubly linked list.
Specifically, we first set p's left pointer to x, and p's right pointer to x's original next node. Then, we update the original next node's left pointer to point to p. Finally, we update x's right pointer to point to p.

📌 中文完整申論答案

在雙向鏈結串列中，當我們要在節點 x 的後面插入新節點 p 時，必須同時更新前後指標，以維持鏈結結構的正確性。假設原本 x 的下一個節點為 y，插入後的順序會變成 x → p → y。首先，將 p 的 left（或 prev）指標設為 x，接著將 p 的 right（或 next）指標設為原本的 y。然後，需將 y 的 left 指標改為指向 p，最後再將 x 的 right 指標改為 p。這四個步驟缺一不可，否則會導致鏈結斷裂或錯誤。因此，插入操作的關鍵在於正確更新四個指標，確保雙向鏈結串列仍然維持正確的前後連結關係。

☞ 插入後：
x → p → 原本的 next
☞ 必做 4 件事：

- p.left = x
- p.right = x.right
- x.right.left = p
- x.right = p

以下是**第 4 題完整整理（英文原題＋中文翻譯＋詳細分析＋答案整合）**，已幫你整理成**期中考可以直接寫的高分版本（完整說明超過百字）**👇

📝 Question 4 (5 pts) — String Pattern Matching

📘 英文題目

The definition of a failure function is as follows:

4. **(5 pts) String Pattern Matching**
The definition of a failure function is as follows:

$$f(j)=\begin{cases} -1, & \text{if } j=0; \\ f^m(j-1)+1, & \text{where } m \text{ is the least int } k \text{ s.t. } p_{f^k(j-1)+1}=p_j; \\ -1, & \text{otherwise.} \end{cases}$$

Note: $f^{-1}(j)=f(j)$, $f^m(j)=f(f^{m-1}(j))$

Complete the failure function of the pattern shown in the following.

j =	0	1	2	3	4	5	6	7	8	9	10	
	a	b	a	c	a	b	a	c	d	b		➔ pattern
f (j) =	-1	?	? …									

Complete the failure function of the pattern shown in the following.

📗 中文翻譯

失敗函數（failure function）的定義如下：

當 j = 0 時，f(j) = -1；
否則，若存在最小整數 k，使得某種 prefix 與 suffix 相等，則 f(j) = f(j-1)+1；
否則 f(j) = -1。

請完成下列 pattern 的 failure function。

🔍 中文詳細分析（超重要🔥 KMP 核心）

這題在考：
☞ **KMP（Knuth-Morris-Pratt）演算法的 failure function（或 LPS）**

□ 什麼是 failure function？

☞ f(j) 的意義：
表示「當 mismatch 發生時，pattern 要跳回哪裡」

換句話說：
☞ **目前位置之前的字串中，最長的 prefix = suffix 的長度（-1 版本）**

□ 題目給的 pattern（第 8 頁）

j: 0 1 2 3 4 5 6 7 8 9 10
pattern: a b a c a b a c d b

□ 計算方式（重點邏輯）

每一個 j 都要做：

☞ 找「最長 prefix = suffix」

例如：

- j=0 → "a" → 沒有 → -1
- j=2 → "aba" → "a" = "a" → 長度 1
- j=5 → "abacab" → "ab" = "ab"

☞ 找的是「最長匹配」

□ 最終答案

f(j): -1 -1 0 -1 0 1 2 3 -1 1

（對應 j = 0~9）

✍ 完整答案（可直接寫）

The failure function of the given pattern is computed by finding the longest proper prefix which is also a suffix for each position in the pattern.

At each position j, we compare the prefix and suffix of the substring from index 0 to j, and record the longest match. If no such prefix exists, the value is -1.

Thus, the resulting failure function is:

f(j): -1 -1 0 -1 0 1 2 3 -1 1

📌 中文完整申論答案

本題要求計算 KMP 演算法中的 failure function。failure function 的意義是在字串比對過程中，當發生 mismatch 時，可以利用已知的 prefix 與 suffix 重複結構來避免重新比對，從而提升效率。對於每一個位置 j，我們都需要找出在子字串 P[0…j] 中，最長的「proper prefix」與「proper suffix」相同的長度。如果不存在這樣的匹配，則記為 -1。透過逐一分析 pattern 中每個位置的 prefix 與 suffix，可以得到對應的 failure function 值。最終結果為 f(j) = -1, -1, 0, -1, 0, 1, 2, 3, -1, 1。此函數是 KMP 演算法的核心，可用來在 mismatch 時快速跳轉，避免不必要的重複比對。

🔥 最後重點（考試秒背）

☞ failure function =
prefix = suffix 的最長長度（不含整段）

☞ 沒有 → -1

☞ KMP 核心用途：
避免回頭比對（O(n+m)）

如果你要，我可以幫你做👇
🔥「這題一步一步動畫版（超直覺）」
🔥「KMP 手算 10 題（老師 100%會考）」

以下是**第 5 題完整整理（英文原題＋中文翻譯＋詳細分析＋答案整合）**，已整理成**期中考可以直接寫的高分版本（完整解析超過百字）**👇

📝 Question 5 (5 pts)

📘 英文題目

How many space is required when you store a polynomial equation $f(x) = 3x^{100} + 4x^{98} + 50x^{30} + 49$ by the representation 1 and the representation 2, respectively?

📗 中文翻譯

當我們使用「表示法一（representation 1）」與「表示法二（representation 2）」來儲存多項式

[

f(x)=3x^{100}+4x^{98}+50x^{30}+49

]

時，請問各自需要多少儲存空間？

Representation 1

```
private:
   int degree;
   float *coef;


Polynomial::Polynomial(int d)
{
        degree = d;
        coef = new float [degree+1];
}
```

Representation 2

```
class Polynomial;

class Term {
friend Polynomial;
private:
   float coef; // coefficient
   int exp; // exponent
};

// The private data members of Polynomial
private:
   Term *termArray; // array of nonzero terms
   int capacity; // size of termArray
   int terms; // # of nonzero terms
```

🔍 中文詳細分析（考試重點🔥）

這題在考：
☞ **多項式的兩種儲存方式（Polynomial Representation）**

□ (1) Representation 1（密集表示法 Dense）

從題目圖（第 9 頁）可知：

```
float* coef;
coef = new float[degree+1];
```

☞ 代表：

- 會建立一個長度 = degree + 1 的陣列
- 每一個指數都要存

📌 套用到題目

最高次數 = 100

☞ 陣列大小 = 100 + 1 = 101

☞ 每個位置存一個係數

再加上：

- degree 本身也要存

✅ 空間計算

- degree：1
- coef array：101

☞ 總空間 = **102 units**

□ (2) Representation 2（稀疏表示法 Sparse）

從題目可知：

```
class Term {
    float coef;
    int exp;
};
```

☞ 只存「非零項」

📌 題目有幾項？

3x^100
4x^98
50x^30
49

☞ 共 4 項

📌 每一項需要

- coef（係數）
- exp（指數）

☞ 每項 = 2 單位

📌 再加上

- terms 數量

- capacity

✅ 空間計算

- capacity：1
- terms：1
- 4 項 × 2 = 8

👉 總空間 = **10 units**

✍ 完整答案（可直接寫）

For representation 1, the polynomial is stored using an array of size (degree + 1). Since the highest degree is 100, the array size is 101, and including the degree itself, the total space required is 102 units.

For representation 2, only nonzero terms are stored. Since there are 4 nonzero terms, and each term stores a coefficient and exponent, the total space required is 4 × 2 = 8 units, plus 2 units for capacity and terms, resulting in a total of 10 units.

📌 中文完整申論答案

本題比較兩種多項式儲存方式的空間使用量。在表示法一（密集表示法）中，會建立一個大小為 degree+1 的陣列，並將所有係數（包含為 0 的項）全部儲存。由於本題最高次數為 100，因此需要 101 個儲存空間來存放係數，再加上 1 個空間儲存 degree，總共需要 102 單位空間。

在表示法二（稀疏表示法）中，只儲存非零項。本題共有 4 個非零項，每一項需要儲存係數與指數，因此需要 4×2=8 個空間，再加上 capacity 與 terms 各 1 個空間，總共為 10 單位空間。

因此，表示法一需要 102 單位，而表示法二只需要 10 單位，顯示稀疏表示法在此情況下更有效率。

🔥 最後重點（考試秒背）

👉 Dense：O(n) 空間（全部存）
👉 Sparse：O(k) 空間（只存非零）

👉 本題答案：

- Representation 1 = **102**
- Representation 2 = **10**

如果你要，我可以幫你做👇
🔥「Polynomial 表示法比較表（期中必考）」
🔥「類似計算題再出 10 題（直接練）」

可以，下面是**第 6 題完整轉換到最後**，包含**每一步 stack 與 output 的變化**，以及**最終 postfix 結果**。題目的運算式是：

[
(A + B);|;(C * D)%E
]

題目英文

Give a table which characterizes the priority of the operators. The expression is (A + B) | (C * D) % E. Please show the status of the stack and output in the first five iterations when you transform the infix expression to the postfix expression in C++ by extracting a token from the equation in serial.

中文翻譯

請列出運算子的優先順序表。已知運算式為
[
(A + B);|;(C * D)%E
]
請用逐一讀取 token 的方式，模擬在 C++ 中將 infix expression 轉成 postfix expression 的過程，並顯示 stack 與 output 的狀態。

```
void Postfix(Expression e) {
// Output the postfix form of the infix expression e.
// NextToken() and stack are as in Eval().
// Assume that the last token in e is '#.' Also, '#' is used at the bottom of the stack
   Stack<Token> stack; // initialize stack
   stack.Push('#');
   for (Token x = NextToken(e); x != '#'; x=NextToken(e))
      if (x is an operand) cout << x;
      else if ( x == ')') { // unstack until '('
         for (; stack.Top() != '('; stack.Pop())
            cout << stack.Top();
         stack.Pop(); // unstack '('
      } else { // x is an operator
         for (; isp(stack.Top()) <= icp(x); stack.Pop())
            cout << stack.Top();
         stack.Push(x)
      }
 // end of expression; empty stack
   for (; !stack.IsEmpty(); cout << stack.Top(), stack.Pop());
    cout << endl;
}
```

priority	operator
1	!
2	*, /, %
3	+, -
4	<, <=, >=, >
5	==, !=
6	&&
7	||

一、運算子優先順序

根據題意，這題會用到的運算子優先順序可整理成：

優先順序	運算子
高	*, %
中	+
低	`

也就是說：
*、% 先做，+ 次之，| 最後。

二、轉換規則簡述

將 infix 轉 postfix 時，規則如下：

1. **遇到運算元**（如 A、B、C、D、E）→ 直接輸出到 output
2. **遇到左括號 (** → push 進 stack
3. **遇到右括號)** → 把 stack 中的運算子一直 pop 到遇到 (為止
4. **遇到運算子** → 比較 stack top 的優先順序
    - 若 stack top 優先順序較高或相等，先 pop 再輸出
    - 否則 push 目前運算子進 stack

三、完整轉換表

原始 infix：

(A+B)|(C*D)%E

下面是**完整逐步轉換表**：

步驟	讀入 Token	動作說明	Stack	Output
1	(	左括號直接	(	

		push		
2	A	運算元直接輸出	(	A
3	+	push 進 stack	(+	A
4	B	運算元直接輸出	(+	AB
5	)	pop 到 (，輸出 +		AB+
6	`	`	stack 空，push `	`
7	(	左括號直接 push	`	(`
8	C	運算元直接輸出	`	(`
9	*	push 進 stack	`	(*`
10	D	運算元直接輸出	`	(*`
11	)	pop 到 (，輸出 *	`	`
12	%	和 stack top `	比較，%` 優先權較高，所以 push	`
13	E	運算元直接輸出	`	%`
14	結束	依序 pop %、`	`	

四、最終 postfix 結果

所以最後的 **postfix expression** 是：

AB+CD*E%|

五、中文詳細分析

這題的重點在於你要理解：**postfix 的順序不是看原本式子從左到右照抄，而是要靠 stack 暫存運算子，等到適當時機再輸出。**
前半段 (A+B) 很單純，因為括號表示 A 和 B 的加法要先完成，所以當讀到) 時，就把 + 從 stack 中彈出，因此得到 AB+。接著遇到 |，因為它是較低優先順序的運算子，所以先暫存在 stack。

後半段 (C*D)%E 中，括號內的 C*D 必須先完成，因此讀到右括號時會先輸出 *，形成 CD*。接著遇到 %，因為 % 的優先順序高於 |，所以 % 直接

push 到 stack 上方，不會先把 | 彈出。再讀到 E 後直接輸出。最後整個輸入結束，stack 裡剩下的 % 和 | 依序彈出，因此最後答案就是 AB+CD*E%|。

換句話說，這個 postfix 正確反映了原本 infix 的運算順序：先做 (A+B)，再做 (C*D)，再做 %E，最後才做 |。

六、考試可直接寫的答案

The complete postfix expression of
[
(A+B)|(C*D)%E
]
is:

AB+CD*E%|

The conversion is done by scanning the infix expression from left to right. Operands are directly appended to the output, while operators are pushed into the stack according to precedence. Parentheses force operators inside them to be popped first. Therefore, after fully processing the expression and emptying the stack, the final postfix expression is AB+CD*E%|.

下面是依照你這張圖的**第 7 題正確版本**，我幫你整理成：

英文題目 + 中文翻譯 + 中文詳細分析 + 完整答案
而且我會把 **(a) count 版程式、(b) 表格分析、最後 Θ 複雜度** 全部放在一起。

第 7 題

英文題目

(10 pts) Please compute the time complexity of [Fibonacci number].
[
F_0 = 0,; F_1 = 1,; \text{and in general } F_n = F_{n-1} + F_{n-2},; n \ge 2.
]

The program Fibonacci inputs any nonnegative integer n and prints the value (F_n).

題目給的程式如下：

```
void Fibonacci (int n)
{// compute the Fibonacci number Fn
1.    if (n <= 1) cout << n << end;      // F0 = 0, F1 = 1
2.    else{
3.          int fn; int fnm2=0; int fnm1=1;
4.          for (int i = 2; i <= n; i++) {
5.                fn = fnm1 + fnm2;    fnm2 = fnm1;    fnm1 = fn;
            }
6.          cout << fn << end;
      }
}
```

中文題目翻譯

（10 分）請計算 Fibonacci number 的時間複雜度。

其中
[
F_0 = 0,; F_1 = 1
]
且一般情況下
[
F_n = F_{n-1} + F_{n-2},; n \ge 2
]

程式 Fibonacci 會輸入任意一個非負整數 n，並輸出 (F_n) 的值。

(a)

請在程式中加入一個新變數 count，並寫出原程式的簡化版本，以便計算它的時間複雜度。

(b)

請建立一個表格，並用漸進符號 **Θ** 算出這個程式的時間複雜度。

中文詳細分析

這題考的不是數學上的 Fibonacci 遞迴公式本身，而是**題目所給這段 C++ 程式的時間複雜度**。這一點很重要，因為很多人一看到 Fibonacci 就直覺想到遞迴版本 (T(n)=T(n-1)+T(n-2))，然後以為是指數時間，但這題給的程式其實**不是遞迴版本**，而是**迴圈版本**。所以這題不能寫成 (O(2^n)) 或指數複雜度，那樣會直接錯掉。

先看程式結構。若 n <= 1，只會做一次判斷，然後直接輸出，這是常數時間。若 n > 1，就進入 else 區塊。第 3 行先宣告並初始化幾個變數，這些都是固定次數操作，所以屬於常數時間。第 4 行有一個 for 迴圈，從 i=2 到 i=n，因此總共會執行 **n−1 次**。而第 5 行每一次迴圈都只做固定數量的指定、加法與賦值，因此每次成本是常數。也就是說，整個演算法最主要的成本來自這個 for 迴圈。

因此，總執行時間可以寫成「常數 + 常數 × (n−1) + 常數」，也就是形如
[
an+b

]
的線性函數。用漸進符號來看，忽略常數與低階項後，得到的時間複雜度就是
[
\Theta(n)
]
。

這題的重點就是要看出：**雖然 Fibonacci 的定義是遞迴式，但這段程式是用迴圈逐步算出結果，所以時間複雜度是線性的，不是指數的。**

(a) 加入 count 的簡化版本

下面是一個可以用來計算步數的簡化版本：

```
void Fibonacci(int n) {
    int count = 0;

    count++;                    // line 1: if 判斷
    if (n <= 1) {
        count++;                // line 1: 輸出
    }
    else {
        count += 3;             // line 3: fn, fnm2=0, fnm1=1
        count++;                // line 4: for 初始化 i=2

        for (int i = 2; i <= n; i++) {
            count++;            // line 4: 條件判斷 i<=n
            count += 3;         // line 5: fn=..., fnm2=..., fnm1=...
            count++;            // line 4: i++
        }

        count++;                // 迴圈最後一次失敗條件判斷
        count++;                // line 6: 輸出 fn
    }
}
```

這個版本的目的不是要真的輸出答案，而是把每個重要操作當成一步來計數。
你在考試時不一定要跟這個版本一模一樣，但核心要表現出來：

1. if 判斷是常數次
2. 初始化變數是常數次
3. for 迴圈做了 n-1 次
4. 每次迴圈內只做常數次操作

(b) 表格分析

下面給你一個很適合考卷書寫的表格版本。

當 n <= 1 時

line	s/e	frequency	total steps
1	1	1	1
1 輸出	1	1	1

所以：
[
t_{Fibonacci}(n)=2 \quad (n \le 1)
]

當 n > 1 時

line	s/e	frequency	total steps
1 if 判斷	1	1	1
3 變數初始化	3	1	3
4 for 初始化	1	1	1
4 for 條件判斷	1	n	n
5 迴圈本體	3	n-1	3(n-1)
4 i++	1	n-1	n-1

line	s/e	frequency	total steps
6 輸出	1	1	1

把它加總：

[
t_{Fibonacci}(n)
= 1 + 3 + 1 + n + 3(n-1) + (n-1) + 1
]

整理：

[
= 1+3+1+n+3n-3+n-1+1
]

[
= 5n + 2
]

所以：

[
t_{Fibonacci}(n)=5n+2
]

因此其時間複雜度為：

[
\Theta(n)
]

最終答案

英文簡答版

The given Fibonacci program is an **iterative** algorithm, not a recursive one.
When (n \le 1), it takes constant time.
When (n > 1), the for loop runs from 2 to n, so it executes (n-1) times.
Each iteration performs only constant-time operations. Therefore, the total running time is linear in (n).

[
t_{Fibonacci}(n)=5n+2
]

Hence, the time complexity is:

[
\Theta(n)
]

中文完整答案版

本題所給的 Fibonacci 程式不是遞迴版本，而是使用迴圈逐步計算 Fibonacci 數列的版本。當 (n \le 1) 時，程式只需要做一次條件判斷與一次輸出，因此時間為常數。當 (n > 1) 時，程式先進行幾個變數初始化，這些都是常數時間；接著進入 for 迴圈，從 i=2 到 i=n，因此總共執行 (n-1) 次。每次迴圈中只包含固定數量的加法與指定動作，所以每次迴圈成本為常數。整體時間可以表示成線性函數，例如
[
t_{Fibonacci}(n)=5n+2
]
忽略常數與低階項後，可得其漸進時間複雜度為
[
\Theta(n)
]
。
因此，第 7 題的正確答案是：**此程式的時間複雜度為 (\Theta(n))**。

你考試可以直接寫的最後一句

$$
\boxed{t_{Fibonacci}(n)=5n+2,\quad \Theta(n)}
$$

如果你要，我下一則可以直接幫你把**第 8 題也照這個格式完整整理**。

下面是**第 8 題完整整理**，包含：

英文題目 + 中文題目翻譯 + 中文詳細分析 + 各小題答案全部整合在一起。我會直接按照你圖片中的 **a~e** 順序幫你完整寫出來，並整理成考試可以直接背、直接寫的版本。

第 8 題

英文題目

8. (15 pts) Circular queue

Two variables front and rear are used to keep track the locations of the first and last elements in the queue which is implemented by an array. Suppose that capacity denote the current size of an array.

a. **(4 pts)** Please identify the locations of front and rear in the following array.
b. **(2 pts)** Please show the equation (line 5) to update the variable rear when a new element is push into the queue (line 7).
c. **(2 pts)** Please show the condition to check if the queue is full (line 1).
d. **(2 pts)** To distinguish it from the condition when the queue is empty, we will increase the capacity before the array is full (lines 2-3). Please give the complexity if we only increase the capacity by one every time the array is full.
e. **(5 pts)** Please give the complexity if we increase the capacity by half each time the array is full and explain it shortly.

中文題目翻譯

第 8 題（15 分）循環佇列 Circular queue

有兩個變數 front 與 rear，用來追蹤以陣列實作的 queue 中，第一個元素與最後一個元素的位置。假設 capacity 表示目前陣列的大小。

a. （**4 分**） 請指出下圖陣列中 front 與 rear 的位置。
b. （**2 分**） 當有一個新元素要 push 進 queue 時，請寫出第 5 行更新 rear 的公式。
c. （**2 分**） 請寫出判斷 queue 是否已滿的條件（第 1 行）。
d. （**2 分**） 為了和 queue 為空的情況區分，在陣列真正滿之前就先擴充容量。若每次滿了只增加 **1 格**，請問其複雜度為何？
e. （**5 分**） 若每次陣列滿了就增加 **原容量的一半**，請問其複雜度為何？並簡要說明。

中文詳細分析

這題在考的是 **circular queue（循環佇列）** 的核心觀念。一般 queue 若用普通陣列來做，當 rear 走到最後面時，即使前面有空位，也可能無法直接使用，因此效率不好。Circular queue 的想法就是把陣列尾端和開頭接起來，讓索引可以透過 % capacity 繞回去，因此能夠重複利用前面空出來的位置。

在這題的圖中，陣列索引從 [0] 到 [7]，內容依序看起來是：
[0]=C, [1]=D, [2]=E, [3]=F, [4]=G, [5]=空, [6]=A, [7]=B。
這代表 queue 的邏輯順序不是單純從左到右，而是從 A 開始，接著 B，再繞回 C, D, E, F, G。因此 front 應該在 A 的前一格位置，而 rear 在最後一個元素 G 所在的位置。這也是很多資料結構課本對 circular queue 的慣用設計：front 指向第一個元素的前一格，rear 指向最後一個元素。

接著，push 新元素時，rear 必須往前移一格，但如果已經在陣列尾端，下一格就要繞回到 0，所以公式一定是模除型式，也就是 (rear + 1) % capacity。判斷 queue 是否已滿時，因為 circular queue 常刻意保留一格不用，避免和空佇列條件混淆，所以滿的條件通常是「若 rear 再往前一格就會撞到 front」，也就是 (rear + 1) % capacity == front。

最後兩題在考 **擴充陣列的成本分析**。如果每次滿了只增加 1 格，那每次都要搬移舊資料，連續很多次下來總成本會變成平方級，因此是 **$O(n^2)$**。但若每次擴充一半，也就是類似動態陣列擴容策略，雖然某些次擴充會花 O(n) 搬移元素，但平均到每次插入後，攤銷成本會降到線性累積，因此每次 push 的 amortized cost 是常數級，而總體可視為 **O(n)**，平均單次為 **O(1)**。

各小題答案

(a) front 和 rear 在哪裡？

由題目圖可判斷：

- A 在 index 6
- B 在 index 7
- C 在 index 0
- D 在 index 1
- E 在 index 2
- F 在 index 3
- G 在 index 4
- index 5 是空的

因此 queue 的邏輯順序是：

A → B → C → D → E → F → G

所以：

- front = 5
- rear = 4

這也和題目後面提供的答案頁一致。

(b) 更新 rear 的公式

當新元素 push 進 circular queue 時，rear 要往前走一格；若走到尾端，則必須繞回開頭，因此公式是：

[
rear = (rear + 1) % capacity
]

也就是：

rear = (rear + 1) % capacity;

這樣不論 rear 原本在中間還是在最後一格，都能正確更新。

(c) queue 滿的條件

因為 circular queue 通常保留一格空位來和「空佇列」區分，所以當 rear 再前進一格就會碰到 front 時，就表示 queue 滿了。條件為：

[
(rear + 1) % capacity == front
]

也就是：

if ((rear + 1) % capacity == front)

這就是第 1 行應該填入的條件。

(d) 每次滿了只增加 1 格，複雜度為何？

如果每次陣列滿了，只把容量增加 1，那麼每擴充一次都要重新配置新陣列，並把原本所有元素複製過去。假設原本有 1 個元素時複製 1 次，2 個元素時複製 2 次，3 個元素時複製 3 次……一直到 n 個元素時複製 n 次，總成本就會變成：

[
1 + 2 + 3 + \cdots + n = O(n^2)
]

所以若每次只增加 1 格，整體複雜度是：

[
O(n^2)
]

這種擴充方式非常沒效率，因為幾乎每次快滿時都要搬移大量資料。

(e) 每次滿了就增加原容量的一半，複雜度為何？

若每次陣列滿了就把容量增加原本的一半，假設容量依序變成：

n, 1.5n, 2.25n, ...

這種成長方式雖然某一次擴充時仍然需要搬移所有舊元素，單次擴充成本是 O(n)，但擴充次數會大幅減少。把所有搬移成本加總後，可得到總成本為線性等級，因此整體插入 n 個元素的總成本是：

[
O(n)
]

若從 amortized analysis 的角度看，則平均每次 push 的成本是：

[
O(1)
]

所以這一題若老師問「complexity」而依題目答案頁，應寫：

[
O(n)
]

因為整體擴充與複製的總工作量是線性的。

最後整理版答案

你可以在考卷上直接寫成下面這樣：

(a) front = 5, rear = 4
(b) rear = (rear + 1) % capacity

(c) (rear + 1) % capacity == front
(d) If capacity is increased by 1 each time, the complexity is O(n^2)
(e) If capacity is increased by half each time, the complexity is O(n)

中文完整作答版（可直接抄）

本題考的是 circular queue 的索引更新、滿佇列判斷，以及擴充容量時的時間複雜度。由圖可知，陣列中元素排列為 C、D、E、F、G、空、A、B，因此 queue 的實際順序應為 A、B、C、D、E、F、G。依 circular queue 的常見定義，front 會放在第一個元素前面一格，因此 front = 5；rear 則指向最後一個元素 G，所以 rear = 4。當新元素加入時，rear 應更新為 (rear + 1) % capacity，因為索引需要能從尾端繞回開頭。判斷 queue 是否已滿時，條件為 (rear + 1) % capacity == front，也就是 rear 再往前一格就會撞到 front。若每次滿了只增加 1 格，則每次擴充都要搬移大量元素，總成本為 (1+2+\cdots+n)，因此複雜度為 (O(n^2))。若每次滿了就增加原容量的一半，雖然擴充當下仍需搬移元素，但擴充次數較少，總搬移成本為線性，因此整體複雜度為 (O(n))，平均每次 push 的攤銷成本可視為 (O(1))。

下面是**第 9 題完整整理**，包含：

英文題目 + 中文翻譯 + 中文詳細分析 + 各小題完整答案
我會依照你圖片中的 **a～d** 一題一題幫你完整寫出來，整理成可以直接拿去背、直接寫在考卷上的版本。

第 9 題

英文題目

9. (15 pts) Matrix stored by circular list.

We can record a sparse matrix by storing only nonzero elements using singly linked lists. The nodes can be divided into two categories: one is a header node, and the other is nonzero element. The information stored in each node is shown in the following figures.

Let av be a static class member of Matrix of type ChainNode<T>* that points to the first node in the chain of deleted nodes.

a. **(2 pts)** Please give the total number of nodes required to record a matrix (M_{4\times 3}) according to this data structure.
b. **(3 pts)** Please explain why the available list av is defined as “static” ChainNode<T>.
c. **(5 pts)** The destructor ~Matrix() returns all nodes to the av list. Please draw a graph to show the sequence of nodes in av. Note that the code has been modified.
d. **(5 pts)** Please modify the code of function GetNode within the underline, which is used to get a node from av if it is nonempty. Note that you have to consider the types of nodes stored in av.

中文題目翻譯

第 9 題（15 分）以 circular list 儲存矩陣

我們可以用 singly linked lists 只儲存 sparse matrix（稀疏矩陣）的非零元素。節點分成兩類：
一類是 **header node**，另一類是 **nonzero element node**。圖中顯示了每種節點裡存放的資訊。

設 av 是 Matrix 類別中的一個 **static 類別成員**，型別為 ChainNode<T>*，它指向被刪除節點所形成串列的第一個節點。

a. 請根據此資料結構，求出記錄矩陣 (M_{4\times3}) 需要的**總節點數**。
b. 請說明為什麼 available list av 要定義成 **static** ChainNode<T>。
c. 解構子 ~Matrix() 會把所有節點歸還到 av list，請畫出 av 中節點的順序圖，注意題目的程式碼已被修改。
d. 請修改 GetNode 函式底線處的程式碼。若 av 非空，就要從 av 取出一個節點。注意：你必須考慮 av 中可能存放的是**不同型態的節點**。

中文詳細分析

這題考的是 **稀疏矩陣（sparse matrix）以 circular linked list 表示** 的設計觀念，同時也結合了 **header node、available list、static class member、destructor 回收節點、以及 node reuse** 等多個資料結構重點。它不是單純背定義，而是要你真正理解這個 linked representation 怎麼運作。

先看這種矩陣表示法的概念。因為 sparse matrix 裡很多元素都是 0，所以如果直接用普通二維陣列會浪費很多空間。因此可以只儲存**非零元素**，每個非零元素節點要記錄 row、col、value，另外還要用 header nodes 來管理列或行的鏈結。題目中這種設計除了 element node 之外，還有 header node，而整體結構需要 headnode，再加上依矩陣維度建立若干 header nodes。

再來，av 是 available list，也就是「可重用節點串列」。當矩陣物件被刪除時，並不是把節點直接丟掉，而是先回收到 av，之後若要建立新矩陣，就可以從 av 取節點再利用。這樣可以減少頻繁 new / delete 的成本。之所以要把 av 宣告成 static，是因為它不是屬於單一某個矩陣物件，而是應該讓**所有 Matrix 物件共同分享**的一塊回收池。如果不是 static，每個物件各自有自己的 av，就無法共享被回收的節點，重用效益也會消失。

第 (c) 題考你讀懂 destructor。它先把 headnode 丟進 av，之後再一列一列把 row 中的節點接回 av。要注意題目說 code has been modified，所以不能照一般

版本亂寫，必須依照題目程式流程推 av 的順序。第 (d) 題則進一步考：因為 av 裡可能同時有 header node 與 element node，所以從 av 取節點時，不能固定只走 right 或只走 down，而要先判斷節點種類。這正是本題最關鍵的細節。

(a) 總共需要多少個 nodes？

題目給的矩陣是：

$$
M_{4\times3}=
\begin{bmatrix}
1 & 0 & 0\\
0 & 5 & 0\\
7 & 0 & 0\\
0 & 0 & 9
\end{bmatrix}
$$

可以看到非零元素共有 4 個，分別是：

- (0,0,1)
- (1,1,5)
- (2,0,7)
- (3,2,9)

這種資料結構所需的節點數量公式是：

$$
\max(\text{rows}, \text{cols}) + \text{非零元素數} + 1
$$

其中：

- rows = 4
- cols = 3
- 非零元素數 = 4
- 最後的 +1 是總 headnode

所以：

$$
\max(4,3)+4+1 = 4+4+1 = 9
$$

答案

$$
\boxed{9 \text{ nodes}}
$$

這也和題目答案頁一致。

(b) 為什麼 av 要定義成 static？

av 是 available list，也就是回收後可再利用的節點串列。若把它定義成 static，表示它屬於整個 class，而不是某一個 Matrix 物件。這樣一來，**所有 Matrix 物件都能共享同一條 av list**。

若不是 static，則每個 Matrix 物件都只擁有自己的 av。那麼 A 物件刪除後回收的節點，就不能被 B 物件拿來再利用，這樣會失去 available list 最主要的目的，也就是**共享已刪除節點、減少記憶體配置成本、提升 reuse 效率**。

答案

Because av should be shared by all Matrix objects.
By defining it as static, all objects of class Matrix use the same pool of recycled nodes, which improves memory reuse and reduces allocation overhead.

(c) destructor 執行後，av 中節點的順序

根據題目後面答案頁與解構子程式的流程，最後 av 的順序是：

[
\boxed{
av \rightarrow (3,2,9) \rightarrow H3 \rightarrow (2,0,7) \rightarrow H2 \rightarrow (1,1,5) \rightarrow H1 \rightarrow (0,0,1) \rightarrow H0 \rightarrow headnode
}
]

也可以用文字畫成：

av
↓
(3,2,9) → H3 → (2,0,7) → H2 → (1,1,5) → H1 → (0,0,1) → H0 → headnode

為什麼是這個順序？

因為 destructor 先把 headnode 接到 av，接著從第一列開始逐列處理。每處理一列，就把該列整串節點回收到 av 前端，再往下一列。因為是「前插法」效果，所以後面處理的列會跑到 av 更前面。因此最終順序會是最後一列的節點最前，依序往前推回去。這就是為什麼最前面會是 (3,2,9)，最後才接到 headnode。

(d) 修改 GetNode 函式

題目要你補的程式，是在 av 非空時從 av 取出一個節點。
關鍵在這句：**you have to consider the types of nodes stored in av**。

也就是說，av 裡可能是：

- header node
- element node

而這兩種 node 在取下一個節點時所用的欄位不同：

- 若是 header node，要沿著 down
- 若是 element node，要沿著 right

因此完整邏輯應該是：

```
template <class T>
ChainNode<T>* Matrix<T>::GetNode()
{ // Provide a node for use
	ChainNode<T>* x;
	if (av) {
		x = av;
		if (av->head) av = av->down;
		else av = av->right;
	}
	else x = new ChainNode<T>;
	return x;
}
```

這也就是題目答案頁上的標準寫法。

全部小題最終答案整理

(a)

[
\boxed{9 \text{ nodes}}
]

(b)

av must be static so that **all Matrix objects share the same available list**, allowing recycled nodes to be reused globally.

(c)

av → (3,2,9) → H3 → (2,0,7) → H2 → (1,1,5) → H1 → (0,0,1) → H0 → headnode

(d)

```
if (av) {
    x = av;
    if (av->head) av = av->down;
    else av = av->right;
}
```

中文完整作答版（可直接抄）

本題考的是以 circular linked list 儲存 sparse matrix 的設計。由於矩陣 (M_{4\times3}) 中只有 4 個非零元素，所以不需要把所有 12 個位置都存起來，只要存非零元素與必要的 header nodes 即可。根據此結構，總節點數公式為 (\max(m,n)+t+1)，其中 (m=4)、(n=3)、(t=4)，因此總節點數為 (\max(4,3)+4+1=9)。available list av 之所以要定義成 static，是因為它應該是整個 Matrix 類別共用的回收節點池，而不是單一物件專屬，這樣所有 Matrix 物件都能共享已刪除節點，提升記憶體重用效率。解構子執行後，所有節點依序回收到 av 中，其順序為 av → (3,2,9) → H3 → (2,0,7) → H2 → (1,1,5) → H1 → (0,0,1) → H0 → headnode。最後，GetNode 若要從 av 取節點，必須先判斷節點型態，若是 header node 就沿 down 取下一個，若是 element node 就沿 right 取下一個，因此程式應寫成 x = av; if (av->head) av = av->down; else av = av->right;。

如果你要，我下一則可以直接幫你做**第 10 題完整版**。